

Coding with AI agents on real systems

What happens when multi agents fight over one codebase

1. Docs are the contract with agents

- Agents read CLAUDE.md, READMEs, comments **at the start of every session**
- If you let agents *write* docs unchecked, the docs drift into agent-flavoured prose
- Successor agents read paraphrased prose, design follow-on work around the paraphrase
- **Read carefully. Edit deliberately. Treat docs like code.**

"Markdown is the new normal"

Tooling: VS Code · *Markdown All in One* · *Markdown Preview Enhanced* · *Marp* (these slides)

2. Capture the *why*, not just the *what*

- **DECISIONS.md** — every non-trivial choice: status, context, decision, **rejected alternatives**
- **/design <feature>** — design doc before coding; captures what you *didn't* build
- **OpenSpec / /opsx:propose** — capability-level specs with REQ-* identifiers
- The diff carries the *what*. Successor agents inherit the file system, not the conversation.

3. Issue $\rightleftharpoons$ PR $\rightleftharpoons$ Agent — strive for 1:1

One Issue	One PR	One Agent session
Focused task	One logical change	Bounded context
Acceptance criteria	Squash-merged	Self-contained brief

Why it matters: reviewable diffs, bounded agent context, clean audit trail (issue → PR → commit → branch deleted).

It's a goal, not religion. Real work spills sometimes.

4. CI is the multi-agent safety net

Concrete defenses shipped in this template:

- **alembic heads** — single-head check catches *sibling migration branches*
- **contract_check.py** — frontend/backend route drift (agent A renames endpoint, agent B's frontend doesn't know)
- **e2e-test runs on single-side PRs** — integration is the cross-cut case
- **Comment-density gate** — pre-commit hook that fails when doc growth outpaces code growth

What passes in isolation can still break main. CI is the gate that catches it.

5. CI has real costs — mitigate, don't disable

Cons

- Slows dev loop (PR-to-merge wall-clock)
- Runner minutes cost real money

Mitigations (this template ships all of them)

- **Scope detection** — single-side PRs skip the other side's suite
- **testmon** — backend tests deselected when their Python deps haven't changed
- **vitest --changed** — frontend tests run only when their module graph intersects the diff
- **Smoke vs slow tier + path triggers** — heavy e2e nightly; pulled onto PR when relevant

Typical small PR: < 2 minutes instead of > 10. Cost down ~5x.

6. Mechanics: where the levers are

- `.claude/skills/` — reusable capabilities. OpenSpec workflow ships here.
- `.claude/commands/*.md` — slash commands. `/review`, `/design`, `/ticket`, `/audit-decisions`.
- **Subagents** — spawn one to handle plumbing. Parent keeps the reasoning thread.
- **Model selection** — opus, sonnet, haiku.
 - Opus: when wrong is silent (math, novel algorithms, hairy refactors)
 - Sonnet: pattern-following plumbing — Pydantic schemas, REST endpoints, fixtures
 - Haiku: cleanups, renames, single-file edits

Opus-for-everything is expensive and often slower than sonnet would have been.

7. Roles and personas

Two ways to frame *which* agent shows up.

Roles — what job it's doing

- junior dev — needs a brief, asks before guessing
- engineering manager — breaks work into issues, doesn't write code
- system architect — proposes design, surfaces tradeoffs
- code reviewer — reads diffs critically, pushes back
- code cleaner — comments, naming, dead code, low-risk

Personas — how it thinks

- meticulous — cross-checks, slow, thorough
- creative — proposes options you didn't list
- **Maverick** — contrarian, breaks the unstated assumption

8. Memory and audit trails

What did the agent actually think six months ago?

- **Conversations are ephemeral. Issues live forever.**
- This template includes:
 - `post-issue-transcript` skill — turns conversation into issue comment
 - `issue-close-reminder.sh` hook — fires on `gh pr merge` / `gh issue close`
 - Memory at `~/.claude/projects/<project>/memory/` — facts that survive sessions

Every closed issue carries the reasoning that produced the change.

9. Supervised autonomy, not autopilot

- `/schedule` runs Claude on cron — polls for `ready`-labelled issues, picks them up
- Sample routines in `.claude/schedules/`:
 - issue-poller · pr-auto-merge · stale-branch-sweep
- **Humans choose the work** (the `ready` label, model selection)
- **Humans approve the merge** (auto-merge on green, or manual review)
- **Agents do the typing. Not the deciding.**